MCA Semester – IV

Interim Report

NovaBank: A Secure Full-Stack Digital Banking Web Application with AI-Inspired Fraud Detection and Financial Health Analytics

Research Project submitted to Jain Online (Deemed-to-be University) in partial fulfillment of the requirements for the award of:

Master of Computer Applications

Submitted by
Chethan BV
USN: 241VMTR01537
Elective: Full Stack Development

Under the guidance of
[Faculty Name - JAIN Online]

Date of Submission: 29/06/2026

DECLARATION

I, Chethan BV, hereby declare that this Interim Report has been prepared by me under the guidance of [Faculty Name - JAIN Online]. I declare that this report is submitted towards the partial fulfillment of the credit requirement for the course “Capstone Project,” which is part of the Master of Computer Applications programme offered by Jain Online. I further declare that the work documented in this report is original in nature and reflects my own contribution.

Place: _____

Date: 29/06/2026

Name of the Student: Chethan BV
USN: 241VMTR01537

EXECUTIVE SUMMARY

NovaBank is a full-stack digital banking web application being developed as a capstone project in Full Stack Development. The project focuses on delivering a secure, role-based banking platform where customers can register, log in, manage beneficiaries, view account details, and perform protected banking operations, while administrators can monitor customer information and platform-wide activity through a dedicated dashboard.

The project uses React, Tailwind CSS, Axios, and React Router on the frontend, with Java Spring Boot, Spring Security, JWT authentication, Spring Data JPA, and MySQL on the backend. During the interim phase, the core authentication flow has been implemented, beneficiary management is operational, and the admin module can retrieve customer details without exposing sensitive fields such as passwords. Additional modules such as secure transfer processing, fraud analytics, transaction history, and financial health insights have also been scaffolded and largely implemented, placing the project ahead of the minimum interim requirement baseline.

This interim report documents the current project scope, methodology, architecture, progress achieved, remaining tasks, and the planned timeline for completion. It also provides evidence that the mandatory interim submission features are functional and aligned with the capstone rubric.

TABLE OF CONTENTS

1. Introduction

The banking domain has rapidly evolved from branch-centric operations to digital-first customer service models. Users now expect seamless access to banking functions such as account management, fund transfer, transaction visibility, and fraud monitoring through secure web-based systems. In response to these expectations, NovaBank is proposed as a secure full-stack digital banking application that demonstrates how modern frontend and backend technologies can be integrated to deliver a production-style banking workflow.

The primary goal of the project is to build a role-based application supporting two actors: Customer and Admin. Customers should be able to register, log in, manage beneficiary payees, access account information, and use banking services through an intuitive interface. Admin users should be able to monitor customer details and platform activity without accessing sensitive information such as passwords. The project is significant because it combines secure authentication, domain modeling, REST API design, relational database implementation, and responsive user interface development within one coherent full-stack system.

From a Full Stack Development perspective, the project is relevant because it demonstrates the complete path from requirement analysis and database design to frontend integration and API security. It also introduces applied concerns such as stateless JWT authentication, layered architecture, DTO-based API design, transactional consistency, and analytics-oriented service modules.

Background and Relevance

Digital banking systems have become central to modern financial service delivery because they reduce dependence on physical branches and enable always-available customer interaction. In the Full Stack Development domain, banking projects are particularly useful academic case studies because they require secure authentication, reliable state handling, database consistency, user-role separation, and responsive user interfaces within one integrated application.

Traditional academic mini-projects often stop at CRUD interfaces, but a banking system requires more than simple data entry. It must protect sensitive user information, maintain auditable transaction records, separate admin and customer privileges, and preserve correctness in financial operations. This makes NovaBank relevant not only as a software product but also as a demonstration of applied software engineering principles.

The project also aligns with broader industry practices in which frontend applications communicate with backend REST APIs using stateless tokens. This pattern is widely used in fintech and SaaS systems because it supports scalable architecture and clear separation

between presentation, business logic, and persistence layers. By using React on the frontend and Spring Boot on the backend, the project mirrors a professional stack used in many enterprise applications.

From an academic perspective, the project is significant because it integrates multiple competencies expected from a full stack learner: frontend development, backend design, API design, security, database normalization, documentation, and report-based communication. The interim report therefore serves not only as a status update but also as evidence that the project is being developed according to a methodical and technically sound plan.

2. Project Scope and Objectives

The scope of the interim phase is focused on establishing the secure foundation of the banking platform and implementing the minimum functional requirements mandated for evaluation. The report therefore concentrates on the authentication workflow, beneficiary management, and admin-side customer visibility, while also documenting the broader architecture that supports later modules.

Within the current scope, the project covers customer registration and login, JWT-based session handling, role-protected routing, customer beneficiary creation and retrieval, and admin access to customer details via safe DTO responses. The broader project architecture also includes account summary, transaction history, fund transfer, fraud analytics, and financial insight modules, although the interim assessment emphasizes the earlier milestone features.

The main scope boundary is that the current project remains a web application and does not include native mobile apps, inter-bank payment gateway integration, or a machine-learning-based fraud engine. The current implementation of beneficiary management stores beneficiary name, bank name, account number, IFSC code, and nickname. A configurable per-beneficiary max limit can be treated as a planned refinement for final-phase alignment with extended banking specifications.

Functional Requirements

- User authentication through register and login workflows.
- Role-based access control for Customer and Admin users.
- Customer profile and account summary retrieval.
- Customer beneficiary creation, update, deletion, and listing.
- Admin view of customer records excluding sensitive password data.
- Transaction and transfer modules prepared for secure banking operations.

- Fraud analytics and financial health insight services prepared for final stage completion.

Non-Functional Requirements

- Security: passwords must be stored using BCrypt hashing and protected APIs must require valid JWT tokens.
- Reliability: database operations related to financial activity should use transactional integrity.
- Maintainability: code should remain modular through layered architecture and DTO-based API contracts.
- Usability: the frontend should remain responsive and easy to navigate for both customer and admin roles.
- Performance: API responses should remain lightweight and avoid exposing unnecessary relational data.

Project Objectives

- Implement a secure customer registration and login workflow using Spring Security, BCrypt, JWT, and React-based authentication state handling.
- Allow customers to add and manage beneficiaries with core payee details required for future banking transactions.
- Allow administrators to view customer details through protected APIs without exposing sensitive fields such as passwords.
- Establish a scalable layered backend architecture using controller, service, repository, dto, entity, config, security, and exception packages.
- Prepare the platform for advanced modules such as transfer processing, transaction history, fraud analytics, and financial health insights in subsequent milestones.

3. Methodology

The project follows an Agile and iterative development methodology. The implementation has been divided into logical increments so that each sprint or milestone establishes a stable feature set before the next one is integrated. This approach is particularly suitable for capstone development because it allows authentication, role-based access, beneficiary management, and admin monitoring to be validated early, while more advanced features are added progressively.

The backend is developed using Java Spring Boot with Spring Security, JWT, Spring Data JPA, and Maven. Spring Boot is used to expose REST APIs, Spring Security enforces authentication and authorization, and JPA simplifies relational data persistence. MySQL is used as the normalized relational database. On the frontend, React with Vite is used to

create a responsive Single Page Application. Tailwind CSS is used for styling, Axios for API consumption, and React Router for navigation and route protection.

The development process also follows layered architecture principles. Controllers handle HTTP requests, services contain business logic, repositories manage data access, DTOs define API payloads, and security classes handle token validation and authentication flow. This structure improves maintainability, readability, and testability.

Table 1. Technology Stack and Frameworks Used

Layer	**Technologies / Tools**
Frontend	React 18, Vite, Tailwind CSS, Axios, React Router
Backend	Java 17, Spring Boot 3, Spring Security, JWT, Spring Data JPA, Maven
Database	MySQL
Documentation & API Testing	Swagger UI via SpringDoc OpenAPI
Design Approach	Layered client-server architecture with modular monolith packaging

Table 2. Development Methodology Phases

Phase	**Description**
Requirement Analysis	Study digital banking workflows, identify interim deliverables, and define customer/admin user roles.
System Design	Design database schema, package structure, frontend routes, and security workflow.
Implementation	Develop APIs, frontend pages, authentication flow, DTOs, and beneficiary/admin features.
Validation	Test registration, login, beneficiary management, and admin visibility using API and UI checks.
Documentation	Prepare synopsis, interim report, diagrams, screenshots, and project presentation artefacts.

4. System Architecture and Design

NovaBank follows a client-server architecture in which the React frontend communicates with a Spring Boot REST backend over HTTP. The backend uses a layered design to separate concerns across controller, service, repository, dto, and security layers. This design improves maintainability and helps ensure that sensitive business logic, such as

authentication and transfer processing, is not mixed with presentation concerns.

The chosen architectural pattern is a modular monolith with enterprise-style layering. This is appropriate for a capstone project because it keeps deployment and reasoning manageable while still demonstrating secure, scalable, and professional software structure. JWT-based stateless authentication is used so that the frontend can call protected backend APIs without relying on server-side sessions.

Key design principles used in the project include separation of concerns, principle of least privilege, DTO-based response shaping, stateless authentication, atomic database transactions for financial operations, and reusable component design on the frontend.

Database Design Overview

The application uses MySQL as a normalized relational database. The main entities are users, roles, accounts, beneficiaries, transactions, and fraud_logs. These entities are mapped through JPA models and linked using one-to-one, many-to-one, and many-to-many relationships as required by the domain model.

At interim stage, the most relevant database flow concerns user creation, role assignment, account provisioning, beneficiary persistence, and admin-safe retrieval of customer records. The design avoids exposing raw entity structures directly to the frontend and instead uses DTOs to return only the required fields.

The database design is intentionally extensible. Although the interim review focuses on login, registration, beneficiary creation, and admin visibility, the same schema already supports transaction recording, transfer processing, and fraud analytics in subsequent phases.

Table 3. Core Database Tables and Purpose

Table	**Purpose**
users	Stores customer/admin identity, contact details, hashed password, and status fields.
roles	Stores role definitions such as ROLE_ADMIN and ROLE_CUSTOMER.
user_roles	Maps users to roles using a normalized many-to-many relation.
accounts	Stores account number, balance, currency, and user linkage.
beneficiaries	Stores payee information such as beneficiary name, account number, bank name, IFSC, and nickname.

Table	Purpose
transactions	Stores ledger-style debit/credit transaction records with sender and receiver account references.
fraud_logs	Stores risk score, risk level, and evaluation reasons linked to transfer events.

Table 4. Major Application Modules

Module	Description
Authentication Module	Handles registration, login, password hashing, token issuance, and protected API access.
Customer Module	Supports profile access, account summary, beneficiary management, and customer-facing operations.
Admin Module	Provides admin dashboard access and customer listing using safe DTO responses.
Beneficiary Module	Stores and validates beneficiary information required for future transfers.
Security Module	Includes JWT filter, custom user details service, CORS setup, and Spring Security configuration.
Persistence Module	Uses JPA repositories and entities to manage MySQL access in a layered way.

Security Design Considerations

Security is a central design concern in NovaBank. Authentication is implemented using Spring Security and JWT so that public endpoints such as registration and login remain separate from protected customer and admin operations. BCrypt password hashing is used to ensure that plain-text passwords are never stored in the database.

The frontend stores the authenticated session payload locally and sends the JWT in the Authorization header on subsequent requests through an Axios interceptor. On the backend, a custom JWT authentication filter validates the token, extracts the username, loads the associated user details, and populates the Spring Security context before controller execution.

Role-based authorization ensures that only admins can access admin endpoints, while customers and admins can access permitted customer routes according to the configured rules. In addition, safe DTOs prevent sensitive entity fields from being exposed accidentally through API responses.

Figure 1. Context Diagram of NovaBank

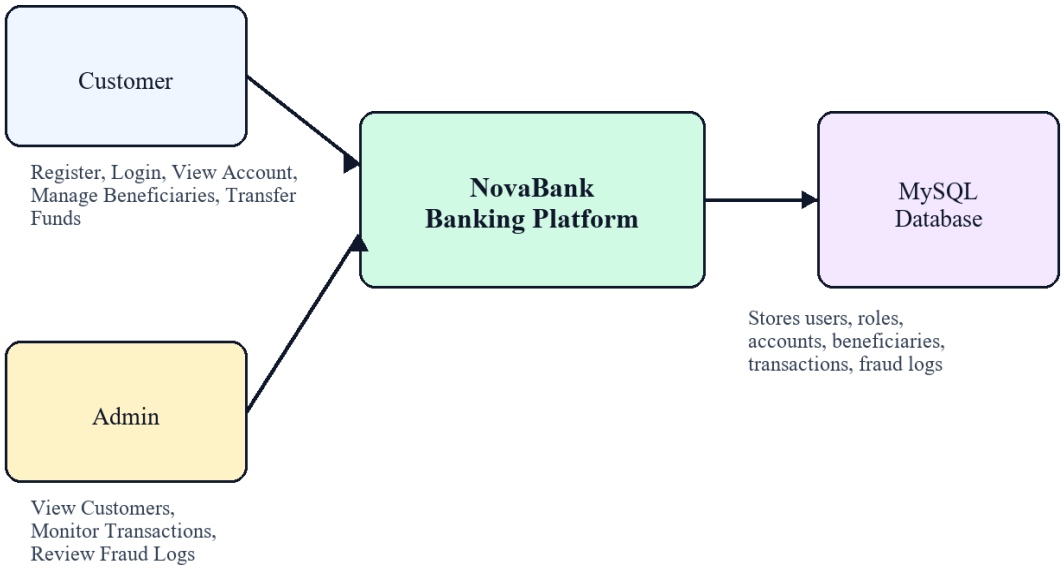

Figure 1. Context Diagram of NovaBank

Figure 2. Layered System Architecture

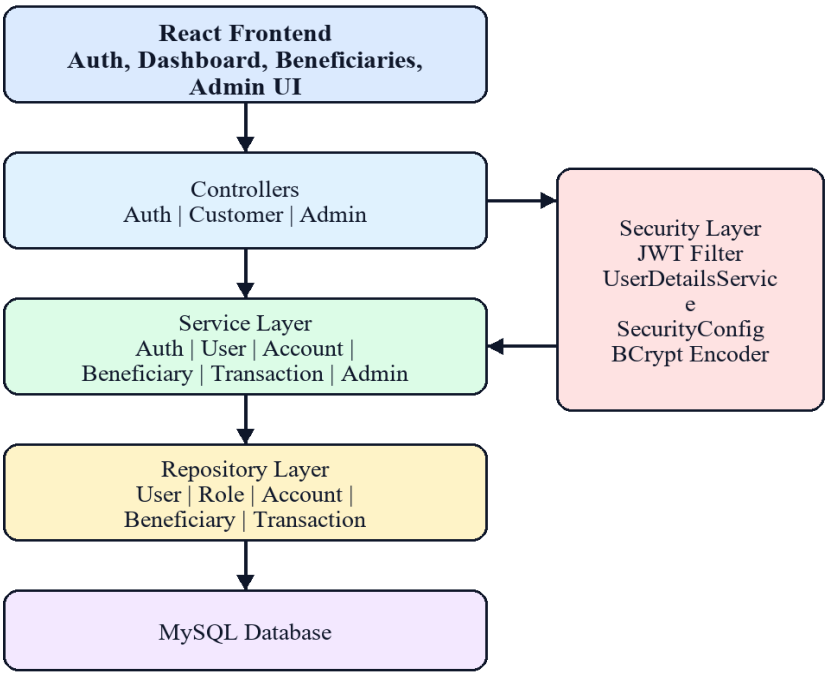

Figure 2. Layered System Architecture

Figure 3. Interim Use Case Diagram

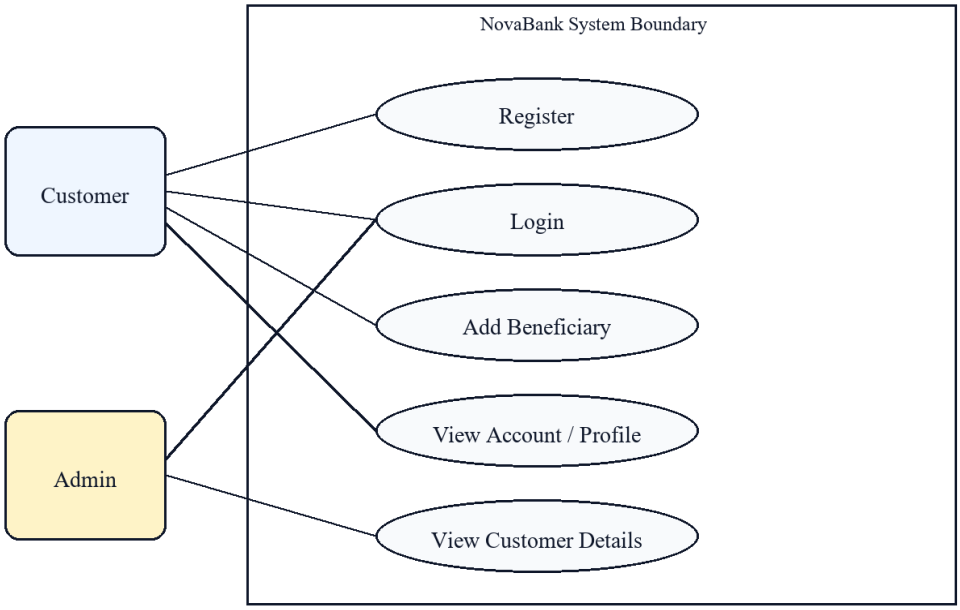

Figure 3. Interim Use Case Diagram

5. Progress and Accomplishments

During the interim period, the project has moved beyond basic setup and now includes a functioning authentication pipeline, customer-facing beneficiary management, and admin visibility into customer data. The implemented codebase contains the full project structure for controllers, services, repositories, entities, DTOs, and security configuration, along with a responsive React frontend and MySQL persistence layer.

The mandatory interim features have been implemented as follows: login is available through the authentication API and React login page, customer registration is available through the registration API and frontend signup page, customer beneficiary creation is implemented through a protected beneficiary creation API and management page, and admin customer viewing is implemented through a protected admin customer listing API that returns only safe profile information. Passwords are excluded because admin responses use DTOs rather than exposing the User entity directly.

Additional progress includes profile management, account summary, transaction history endpoints, dashboard routes, transfer processing logic, fraud log generation, and financial insight calculations. This means the project already has a stronger functional base than the minimum interim checklist, although not all features need to be emphasized equally during interim evaluation.

The main challenge encountered so far has been aligning the banking data model with safe transactional behavior while still keeping the architecture simple enough for academic presentation. Another challenge has been schema evolution during iterative development, especially when changes in transfer ledger design required corresponding updates in database constraints.

Table 2. Interim Progress Matrix

Milestone	**Status**	**Remarks**
Project scaffolding and repository setup	Completed	Backend and frontend structures initialized with clear package separation.
User registration	Completed	Public registration API creates customer user and account.
User login with JWT	Completed	Spring Security and React auth flow are operational.
Customer beneficiary addition	Completed	Protected create/list/update/delete beneficiary APIs implemented.
Admin view customer details	Completed	Protected admin customer API returns safe profile DTOs without password exposure.
Account summary and profile module	Completed	Customer profile and account endpoints implemented.
Transfer and transaction modules	In progress / advanced build	Core logic implemented; final validation and reporting alignment ongoing.
Fraud analytics and financial insights	In progress / advanced build	Service and UI modules prepared for final polish.

Table 3. Mandatory Interim Functionality Coverage

Functionality	**Coverage**	**Evidence**
Login	Implemented	Available through /api/auth/login and React Login page.
Register / Signup	Implemented	Available through /api/auth/register and React Register page.
Customer can add beneficiaries	Implemented	Protected beneficiary API and management page are operational.
Admin can view customer details excluding password	Implemented	Admin uses UserProfileDto and does not receive password data.

Testing and Validation Evidence

During the interim phase, validation has focused on functionality testing and API correctness rather than full automated test coverage. The registration and login flows have been tested through the frontend pages as well as REST endpoint validation. Beneficiary creation and retrieval have been verified as protected customer operations, and the admin customer listing has been checked to ensure that password fields are not returned.

Swagger UI and application-level verification have been used as practical evidence tools during development. Error handling paths such as duplicate email or phone registration and unauthorized route access have also been considered in the backend design through validation annotations and centralized exception handling.

User Interface Evidence

The interim frontend already includes distinct pages for login, registration, customer dashboard, beneficiary management, transaction history, transfer, fraud analytics, financial insights, and admin dashboard routing. For interim assessment, the most relevant user-interface evidence lies in the login page, register page, beneficiary management page, and admin customer visibility page.

The UI design uses React components and Tailwind CSS to provide a modern and responsive layout. Route protection on the frontend complements backend authorization by ensuring that unauthenticated users are redirected to login and that role-based navigation remains user-friendly.

Table 4. Challenges Encountered During Interim Phase

Challenge	**Observation**
Schema evolution	Changes in transfer ledger design required care when aligning entity structure and database constraints.
Role-safe API design	Customer details had to be exposed to admin users without leaking password or internal entity information.
State synchronization	Frontend auth state, route protection, and backend token validation had to remain consistent.
Documentation alignment	The report and diagrams needed to reflect implemented scope honestly while remaining academically complete.

6. Future Work and Timeline

The remaining work for project completion is centered on final stabilization, broader testing, UI refinement, and documentation polish. The secure transfer workflow, analytics modules, and admin dashboard foundation are already in place, but the final phase should

focus on validation, completeness, and presentation readiness.

Potential risks during the remaining phase include integration defects between modules, database migration mismatches when entity structures evolve, user interface edge cases on smaller devices, and incomplete audit of all protected route scenarios. Another practical risk is time compression near final submission, which can reduce the attention given to testing and report quality unless managed carefully.

To mitigate these risks, the remaining work should prioritize regression testing, schema verification, focused bug fixing, and documentation synchronization between the codebase and final report artefacts.

Table 5. Remaining Work Timeline

Week	**Planned Activities**
Week 1	Requirement analysis, project planning, repo setup, initial backend and frontend scaffolding
Week 2	Database schema design, entity modeling, repository creation, datasource configuration
Week 3	Authentication module, JWT flow, role-based security, registration and login pages
Week 4	Beneficiary module, account/profile APIs, admin customer listing, exception handling
Week 5	Transaction processing, atomic update design, fraud log integration
Week 6	Frontend integration, dashboards, protected routing, transaction views
Week 7	Testing, bug fixing, responsiveness review, diagram and report refinement
Week 8	Final stabilization, viva preparation, final report and presentation submission

Table 6. Risk Assessment for Remaining Phase

Risk	**Possible Impact**	**Mitigation**
Integration inconsistency	Mismatch between frontend payloads and backend DTO expectations	Keep API contracts documented and revalidate key screens after every backend update.
Schema mismatch	Database constraints may lag behind entity changes	Review schema after entity changes and test high-risk flows such as transfer and beneficiary operations.

Risk	Possible Impact	Mitigation
Security gap	Protected routes may not be uniformly validated across modules	Retest JWT flow and role restrictions for all customer/admin endpoints.
Time pressure	Documentation and testing may be rushed near final submission	Freeze feature scope and reserve dedicated time for final polishing and verification.

ANNEXURE A. KEY IMPLEMENTED ENDPOINTS

The following endpoints provide concise evidence of the minimum interim functionality that has been implemented in the current project version.

Table 7. Key Implemented REST Endpoints

Endpoint	Purpose
POST /api/auth/register	Public customer registration
POST /api/auth/login	Authentication and JWT token issuance
POST /api/customer/beneficiaries	Add beneficiary for logged-in customer
GET /api/customer/beneficiaries	View beneficiary list for logged-in customer
GET /api/admin/customers	Admin view of customer details using safe DTOs

ANNEXURE B. API SUMMARY AND DATABASE OVERVIEW

This annexure summarizes key API routes and database mappings that support the implemented interim functionality.

Table 8. API Summary

Endpoint	Access	Purpose
POST /api/auth/register	Public	Customer signup and account provisioning
POST /api/auth/login	Public	Credential verification and JWT token generation
GET /api/customer/profile	Customer/Admin	Fetch user profile details
GET /api/customer/account	Customer/Admin	Fetch customer account summary
POST /api/customer/beneficiaries	Customer/Admin	Create a new beneficiary
GET /api/customer/beneficiaries	Customer/Admin	List saved beneficiaries
GET /api/admin/customers	Admin	View customer details without sensitive password data

Table 9. Entity-to-Module Mapping

Entity Group	Mapped Module
User + Role	Authentication and Authorization
Account	Profile and Account Summary
Beneficiary	Beneficiary Management
Transaction	Transfer and History Modules
FraudLog	Fraud Analytics

ANNEXURE C. ADDITIONAL DESIGN DIAGRAMS

The following diagrams provide additional technical explanation for database relationships, authentication flow, user activity, and deployment view.

Figure 4. Database Entity Relationship View

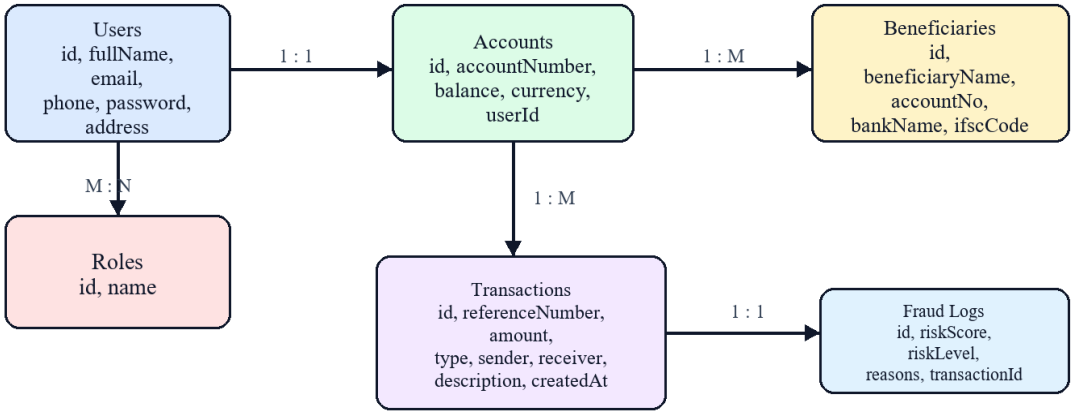

Figure 4. Database Entity Relationship View

Figure 5. Authentication Sequence Diagram

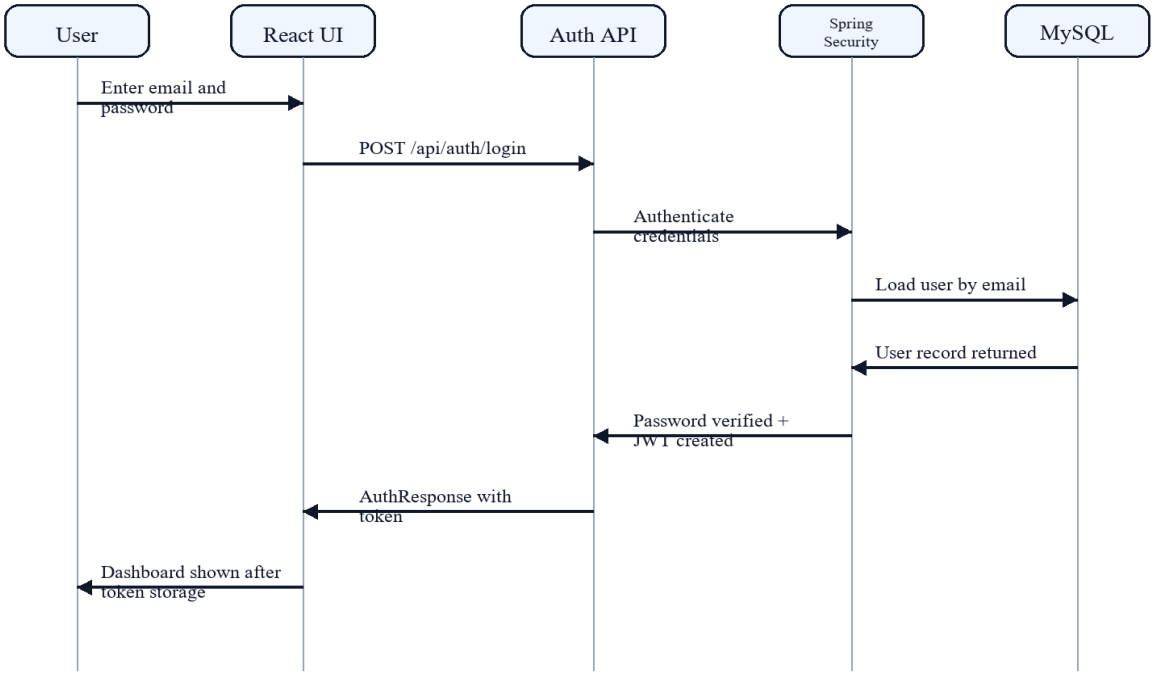

Figure 5. Authentication Sequence Diagram

Figure 6. Beneficiary Management Activity Diagram

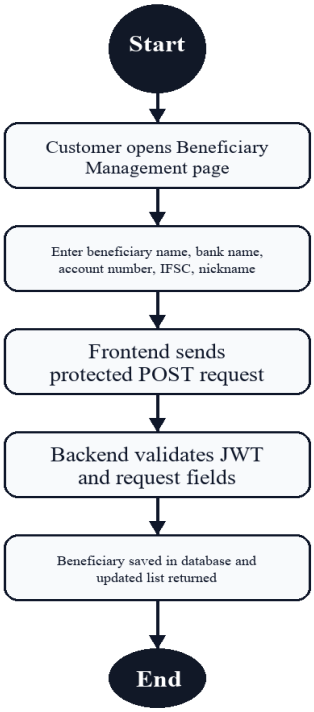

Figure 6. Beneficiary Management Activity Diagram

Figure 7. Deployment / Runtime View

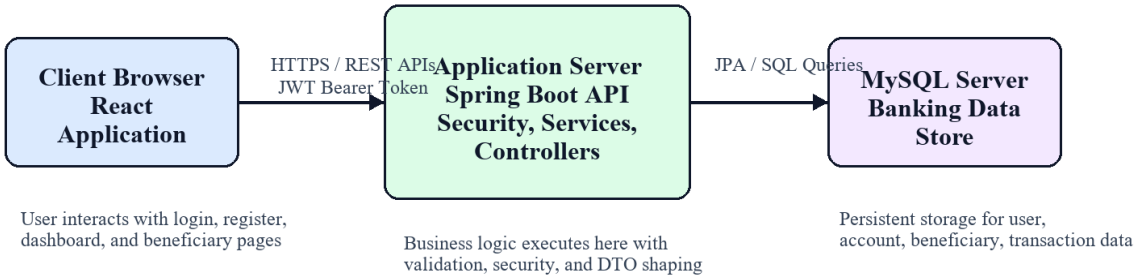

Figure 7. Deployment / Runtime View

7. Conclusion

The interim phase of NovaBank demonstrates meaningful progress toward a full-stack banking platform. The project now includes the core security and role-management foundation required for a digital banking application, and it satisfies the central interim deliverables of registration, login, beneficiary creation, and admin-side customer visibility.

Important lessons learned so far include the value of DTO-based API design for security, the importance of maintaining clean service-layer boundaries, and the need to keep database evolution synchronized with business logic. Adjustments have also been made in the transaction model to better reflect sender-receiver ledger behavior and to preserve consistency during fund transfer operations.

The next steps are to complete final validation of remaining modules, finish documentation and presentation artefacts, and strengthen the overall polish of the project. By the end of the capstone, NovaBank is expected to stand as a coherent, secure, and professionally structured full-stack banking application.